Propositional Calculus (حساب القضايا)

📌 القسم الأول: الرموز والجمل (Symbols & Sentences)

🔷 أنواع الرموز في Propositional Logic

النوع	الرموز	الأمثلة	الوصف
Propositional Symbols	P, Q, R, S	P: "السماء زرقاء"	جمل خبرية تأخذ قيمة T أو F
Truth Symbols	true, false	-	قيم ثابتة
Logical Connectives	∧, ∨, ¬, →, ≡	-	روابط منطقية

🔷 الروابط المنطقية (Logical Connectives)

الرمز	الاسم	المعنى	بالعربي
¬	Negation	NOT	النفي
∧	Conjunction	AND	و
∨	Disjunction	OR	أو
→	Implication	IMPLIES	يستلزم
≡ (أو ↔)	Equivalence	IFF (if and only if)	التكافؤ

🔷 قواعد تكوين الجمل (Well-Formed Formulas - WFFs)

✅ الجمل الصحيحة:

1. **كل رمز قضية جملة:**

   `P`، `Q` → جمل صحيحة

2. **true و false جمل:**

   `true`، `false` → جمل صحيحة

3. **النفي:**

   إذا كانت P جملة، فـ `¬P` جملة

4. **الوصل (Conjunction):**

   إذا كانت P و Q جمل، فـ `(P ∧ Q)` جملة

5. **الفصل (Disjunction):**

   إذا كانت P و Q جمل، فـ `(P ∨ Q)` جملة

6. **الاستلزام (Implication):**

   إذا كانت P و Q جمل، فـ `(P → Q)` جملة

   - **P:** المقدمة (Premise/Antecedent)
   - **Q:** النتيجة (Conclusion/Consequent)

7. **التكافؤ (Equivalence):**

   إذا كانت P و Q جمل، فـ `(P ≡ Q)` جملة

📌 القسم الثاني: الدلالات (Semantics)

🔷 التفسير (Interpretation)

التفسير: تخصيص قيمة صدق (T أو F) لكل رمز قضية

مثال:

```text
P = T  (المطر نازل)
Q = F  (الشمس طالعة)
```

🔷 جداول الحقيقة (Truth Tables)

1. النفي (¬)

P	P¬
T	F
F	T

2. الوصل (∧) - AND

P	Q	P ∧ Q
T	T	**T**
T	F	F
F	T	F
F	F	F

القاعدة: صحيح فقط إذا **كلا الطرفين** صحيح

---

3. الفصل (∨) - OR

P	Q	P ∨ Q
T	T	T
T	F	T
F	T	T
F	F	**F**

القاعدة: خاطئ فقط إذا **كلا الطرفين** خاطئ

4. الاستلزام (→) - IMPLIES ⚠️ مهم جداً

P	Q	P → Q
T	T	T
T	F	⚠️ F
F	T	T
F	F	T

القاعدة: خاطئ فقط إذا المقدمة **صحيحة** والنتيجة **خاطئة**

ملاحظة مهمة:

- ✅ `True = (أي شيء → False)`
- ❌ `False = (True → False)`

---

5. التكافؤ (≡) - EQUIVALENCE

P	Q	P ≡ Q
T	T	T
T	F	F
F	T	F
F	F	T

القاعدة: صحيح إذا **الطرفان لهما نفس القيمة**

📌 القسم الثالث: التكافؤ المنطقي (Logical Equivalence)

🔷 تعريف التكافؤ

تعبيران متكافئان إذا كان لهما **نفس قيمة الصدق** في جميع الحالات الممكنة.

رمز التكافؤ: `≡`

🔷 قوانين التكافؤ المهمة (Logical Identities)

1️⃣ قانون دي مورجان (De Morgan's Law) 🔥

```text
¬(P ∨ Q) ≡ (¬P ∧ ¬Q)
¬(P ∧ Q) ≡ (¬P ∨ ¬Q)
```

بالعربي:

- نفي "و" = "أو" مع نفي الطرفين
- نفي "أو" = "و" مع نفي الطرفين

---

2 قانون التضاد (Contrapositive Law) 🔥

```text
(Q → ¬P¬) ≡ (P → Q)
```

مثال:

- "إذا كان الجو ممطر، الأرض مبتلة"
- **يساوي:** "إذا لم تكن الأرض مبتلة، الجو ليس ممطر"

---

3 قوانين الاستلزام (Implication Laws)

```text
(P ∨ Q¬) ≡ (P → Q)
```

مهم جداً: يمكن تحويل Implication إلى Disjunction!

---

4 قوانين التبادل (Commutative Laws)

```text
(Q ∧ P) ≡ (P ∧ Q)
(Q ∨ P) ≡ (P ∨ Q)
```

5 قوانين الدمج (Associative Laws)

```text
(P ∧ (Q ∧ R)) ≡ (R ∧ (P ∧ Q))
(P ∨ (Q ∨ R)) ≡ (R ∨ (P ∨ Q))
```

---

6 قوانين التوزيع (Distributive Laws)

```text
P ∧ (Q ∨ R) ≡ (P ∧ Q) ∨ (P ∧ R)
P ∨ (Q ∧ R) ≡ (P ∨ Q) ∧ (P ∨ R)
```

---

7 قوانين النفي المزدوج (Double Negation)

```text
P ≡ (P¬)¬
```

8️⃣ قوانين الهوية (Identity Laws)

```text
P ∧ true ≡ P
P ∨ false ≡ P
P ∧ false ≡ false
P ∨ true ≡ true
```

📌 القسم الرابع: أنواع الجمل المنطقية

🔷 التصنيفات الأساسية

النوع	الوصف	مثال
Tautology (Validity)	صحيح في **جميع** الحالات	`P ∨ ¬P`
Contradiction (Unsatisfiable)	خاطئ في **جميع** الحالات	`P ∧ ¬P`
Contingency (Satisfiable)	صحيح في **بعض** الحالات فقط	`P ∧ Q`

📝 أمثلة محلولة

مثال 1: إثبات تكافؤ باستخدام جدول الحقيقة

أثبت أن: `(P ∨ Q¬) ≡ (P → Q)`

P	Q	P¬	P → Q	P ∨ Q¬
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

النتيجة: العمودان متطابقان → **التعبيران متكافئان** ✅

مثال 2: تبسيط باستخدام قوانين دي مورجان

بسّط: `¬(R ∨ (P ∧ Q))`

مثال 3: تحويل Implication

حوّل إلى صيغة بدون →: `(Q → R) ∧ (P → Q)`

الحل:

```text
(Q → R) ∧ (P → Q)
≡ (¬P ∨ Q) ∧ (¬Q ∨ R)    [تطبيق: P → Q ≡ ¬P ∨ Q]
```

🎯 ملخص سريع للحفظ

جدول الحقيقة السريع:

```text
P ∧ Q → صحيح فقط إذا الاثنان صحيح
P ∨ Q → خاطئ فقط إذا الاثنان خاطئ
P → Q → خاطئ فقط إذا P صحيح و Q خاطئ
P ≡ Q → صحيح إذا القيمتان متساويتان
```

القوانين الأساسية:

De Morgan's: ¬(P ∧ Q) ≡ (¬P ∨ ¬Q)

Contrapositive: (P → Q) ≡ (¬Q → ¬P)

Implication: (P → Q) ≡ (¬P ∨ Q)

Predicate Calculus (حساب المحاولات)

📌 مقدمة: الفرق بين Propositional و Predicate Calculus

الخاصية	Propositional Logic	Predicate Calculus
التعامل مع الجملة	ككتلة واحدة (P، Q)	تفصيل داخلي للمكونات
التعبير عن العلاقات	❌ لا يمكن	✅ يمكن
المتغيرات	❌ غير موجودة	✅ موجودة (X, Y)
Quantifiers	❌ غير موجودة	✅ موجودة (∀, ∃)
مثال	`P` : "جورج يحب كيت"	`likes(george, kate)`

الفائدة الأساسية: Predicate Calculus يسمح بالتعميم والتعبير عن علاقات معقدة!

📌 القسم الأول: الرموز (Symbols)

🔹 قواعد تسمية الرموز (Syntax Rules)

القاعدة	التفاصيل
الحروف المسموحة	حروف إنجليزية (A-Z, a-z)، أرقام (0-9)، الشرطة السفلية (_)
بداية الرمز	يجب أن يبدأ بحرف (ليس برقم)
ممنوع استخدام	مسافات، رموز خاصة (*, &, %)، أقواس، فواصل

🔹 الأنواع الأربعة للرموز

1️⃣ الثوابت (Constants)

الوصف	الشرط	أمثلة
تدل على كائن محدد	تبدأ بحرف **صغير**	`george`, `kate`, `tree`, `red`

أمثلة:

- `john` → شخص محدد
- `baghdad` → مدينة محددة

2 المتغيرات (Variables)

الوصف	الشرط	أمثلة
تدل على فئة عامة	تبدأ بحرف **كبير**	`X`, `Y`, `Person`, `City`

أمثلة:

- `X` → أي شخص
- `Y` → أي مدينة
- `Object` → أي كائن

---

3 الدوال (Functions)

الوصف	الشرط	أمثلة
تربط عناصر وترجع قيمة	تبدأ بحرف **صغير**	`father(X)`, `plus(2,3)`, `age(john)`

Arity: عدد المعاملات (Arguments)

Function	Arity	المعنى
`father(david)`	1	والد ديفيد
`plus(2, 3)`	2	جمع 2 + 3
`distance(X, Y, Z)`	3	المسافة بين ثلاث نقاط

مهم: الدالة ترجع **كائن/قيمة**، ليس جملة!

---

4 المحمولات (Predicates)

الوصف	الشرط	أمثلة
توصف علاقة بين كائنات	تبدأ بحرف **صغير**	`likes(X, Y)`, `near(X, Y)`, `red(X)`

Arity: عدد الأطراف في العلاقة

Predicate	Arity	المعنى
`red(ball)`	1	الكرة حمراء (خاصية)
`likes(george, kate)`	2	جورج يحب كيت (علاقة ثنائية)
`between(X, Y, Z)`	3	X بين Y و Z

مهم: المحمول يرجع **جملة** (True أو False)!

🔥 الفرق المهم: Function vs Predicate

الخاصية	Function	Predicate
الناتج	كائن/قيمة	جملة (T/F)
الاستخدام	داخل Term	يكوّن Sentence
مثال	`father(david)` → شخص	`father_of(john, david)` → جملة

مثال توضيحي:

- `father(david)` → **الشخص** الذي هو والد ديفيد
- `father_of(john, david)` → **حقيقة**: جون هو والد ديفيد (T/F)

📌 القسم الثاني: المصطلحات (Terms)

🔷 تعريف Term

Term: تعبير يشير إلى كائن في المسألة

🔷 أنواع Terms

النوع	مثال
Constant	`john`, `5`, `baghdad`
Variable	`X`, `Y`, `Person`
Function Expression	`father(john)`, `plus(2, 3)`, `age(X)`

أمثلة:

```text
✅ john          → ثابت
✅ X             → متغير
✅ father(david) → دالة
✅ plus(age(X), 5) → دالة متداخلة
```

ملاحظة: كل Term يشير لكائن، **ليس جملة!**

📌 القسم الثالث: الجمل (Sentences)

🔷 الجمل الذرية (Atomic Sentences)

تعريف: أبسط نوع من الجمل، تتكون من **Predicate + Terms**

الصيغة:

```text
predicate_name(term1, term2, ..., termN)
```

أمثلة:

الجملة	المعنى
`likes(george, kate)`	جورج يحب كيت
`red(ball)`	الكرة حمراء
`near(X, baghdad)`	X قريب من بغداد
`parent(father(john), john)`	والد جون هو أب لجون

حالات خاصة:

- `true` → جملة ذرية
- `false` → جملة ذرية

◆ الجمل المركبة (Complex Sentences)

يمكن ربط الجمل بالروابط المنطقية:

الرابط	المثال
∧	`likes(john, mary) ∧ likes(mary, john)`
∨	`red(X) ∨ blue(X)`
¬	`likes(george, rain)¬`
→	`student(X) → studies(X)`
≡	`sibling(X,Y) ≡ sibling(Y,X)`

📌 القسم الرابع: المكممات (Quantifiers) 🔥

◆ أنواع Quantifiers

1️⃣ المكمم الكلي (Universal Quantifier) - ∀

المعنى: "لكل"، "مهما كانت"، "جميع"

الصيغة:

```text
∀X sentence(X)
```

أمثلة:

الجملة	المعنى
`∀X likes(X, ice_cream)`	**الجميع** يحبون الآيس كريم
`∀X (human(X) → mortal(X))`	**كل** إنسان فانٍ
`∀X ∀Y (parent(X,Y) → loves(X,Y))`	**كل** أب يحب ابنه

2 المكمم الوجودي (Existential Quantifier) - ∃

المعنى: "يوجد"، "هناك"، "على الأقل واحد"

الصيغة:

```text
X sentence(X)∃
```

أمثلة:

الجملة	المعنى
`Y friends(Y, peter)∃`	**يوجد** شخص صديق لبيتر
`X (doctor(X) ∧ smart(X))∃`	**هناك** دكتور ذكي
`X ∀Y likes(X, Y)∃`	**يوجد** شخص يحب **الجميع**

◆ الفرق بين ∀ و ∃

المكمم	الترجمة	متى نستخدمه
X∀	**لكل** X	عبارات عامة تنطبق على الجميع
X∃	**يوجد** X	عبارات خاصة (على الأقل واحد)

◆ ترتيب Quantifiers مهم! ⚠️

الجملة	المعنى
`X ∃Y likes(X, Y)∀`	**كل شخص** يحب **شخصاً ما** (قد يكون مختلف)
`Y ∀X likes(X, Y)∃`	**يوجد شخص الجميع** يحبونه

مثال توضيحي:

- `X ∃Y mother(Y, X)∀` → **كل شخص** له **أم** (صحيح)
- `Y ∀X mother(Y, X)∃` → **يوجد شخص** هو **أم للجميع** (خاطئ!)

📌 القسم الخامس: قواعد بناء الجمل (WFF Rules)

✅ جملة صحيحة (Well-Formed Formula) إذا:

1. Atomic Sentence: `predicate(term1, term2, ...)`
2. **true** أو **false**
3. **¬S** إذا كانت S جملة
4. **S1 ∧ S2** إذا كانت S1 و S2 جمل
5. **S1 ∨ S2** إذا كانت S1 و S2 جمل
6. **S1 → S2** إذا كانت S1 و S2 جمل
7. **S1 ≡ S2** إذا كانت S1 و S2 جمل
8. **∀X S(X)** إذا كانت S(X) جملة
9. **∃X S(X)** إذا كانت S(X) جملة

📝 أمثلة محلولة

مثال 1: تحديد نوع كل رمز

الجملة: `likes(father(john), ice_cream)`

الرمز	النوع	السبب
`likes`	Predicate (arity 2)	يبدأ بحرف صغير، يكوّن جملة
`father`	Function (arity 1)	يبدأ بحرف صغير، يرجع كائن
`john`	Constant	يبدأ بحرف صغير، كائن محدد
`ice_cream`	Constant	كائن محدد

مثال 2: ترجمة من العربية إلى **Predicate Calculus**

الجملة: "كل الطلاب يدرسون"

الحل:

```text
∀X (student(X) → studies(X))
```

الشرح:

- `∀X` → لكل X
- `student(X)` → إذا كان X طالباً
- `→` → إذن
- `studies(X)` → X يدرس

مثال 3: الفرق بين **Function** و **Predicate**

السؤال: ما الفرق بين:

- `father(john)`
- `father_of(john, david)`

الجواب:

التعبير	النوع	الناتج	المعنى
`father(john)`	Function	**كائن** (شخص)	والد جون
`father_of(john, david)`	Predicate	**جملة** (T/F)	جون والد ديفيد؟

مثال 4: تحليل جملة معقدة

الجملة: `∀X (human(X) → ∃Y loves(X, Y))`

الترجمة:

- "كل إنسان يحب شخصاً ما"

التفصيل:

- `∀X` → لكل X
- `human(X)` → إذا كان X إنساناً
- `→` → إذن
- `∃Y` → يوجد Y
- `loves(X, Y)` → X يحب Y

🎯 ملخص سريع للحفظ

الرموز الأربعة:

```text
Constants:  john, baghdad      [lowercase]
Variables:  X, Y, Person       [Uppercase]
Functions:  father(X), plus(2,3) [lowercase, returns object]
Predicates: likes(X,Y), red(X)   [lowercase, returns T/F]
```

Quantifiers:

```text
∀X → "لكل" → عبارات عامة
∃X → "يوجد" → عبارات خاصة
```

Term vs Sentence:

```text
Term     → يشير لكائن
Sentence → عبارة خبرية (T/F)
```

Semantics of Predicate Calculus (دلالات حساب المحمولات)

📌 مقدمة: لماذا نحتاج Semantics؟

في Propositional Logic	في Predicate Calculus
نعطي قيمة T/F للجملة كاملة	نربط الرموز بأشياء حقيقية في العالم
`P = True`	`george` → شخص محدد في Domain
بسيط ومباشر	معقد ويحتاج Domain of Discourse

Semantics = المعنى الفعلي للرموز والجمل

📌 القسم الأول: التفسير (Interpretation)

🔷 تعريف Interpretation

التفسير: عملية ربط الرموز بأشياء حقيقية في مجال محدد (Domain)

مكونات Interpretation:

المكون	الوصف	مثال
Domain (D)	مجموعة الأشياء المتاحة	`{john, mary, kate, george}`
Constant mapping	تخصيص كل ثابت لعنصر في D	`george` → الشخص جورج
Function mapping	تحديد ناتج الدالة لكل معامل	`father(john)` → george
Predicate mapping	تحديد العلاقات الصحيحة	`likes(george, kate)` → True

🔷 مثال تطبيقي: Blocks World

Domain:

```text
D = {a, b, c, table}
```

:Constants

```text
block_a → a
block_b → b
block_c → c
```

:Functions

```text
top(a) → b      (الكتلة b فوق a)
top(b) → c      (الكتلة c فوق b)
```

:Predicates

```text
on(b, a) → True      (b فوق a)
on(c, b) → True      (c فوق b)
on(a, table) → True (a على الطاولة)
on(c, a) → False     (c ليست مباشرة فوق a)
```

📌 القسم الثاني: تحديد قيم الصدق (Truth Values)

🔷 الجمل الذرية (Atomic Sentences)

القاعدة: نتحقق من التفسير مباشرة

الجملة	التفسير	النتيجة
`likes(george, kate)`	العلاقة موجودة في التفسير	True
`likes(george, john)`	العلاقة غير موجودة	False
`on(b, a)`	b فوق a (حسب التفسير)	True

🔷 الروابط المنطقية

نفس قواعد Propositional Logic:

الرابط	القاعدة
`¬S`	عكس قيمة S
`S1 ∧ S2`	True إذا كلاهما True
`S1 ∨ S2`	False إذا كلاهما False
`S1 → S2`	False إذا S1=True و S2=False

🔷 المكممات (Quantifiers) 🔥

المكمم الكلي (∀) - Universal Quantifier

القاعدة:

الجملة `∀X P(X)` صحيحة **فقط إذا** كانت P(X) صحيحة **لجميع** قيم X في Domain

```
Domain: {john, mary, kate}
Predicate: human(X)
∀X human(X) → True  إذا: human(john)=T ∧ human(mary)=T ∧ human(kate)=T
∀X human(X) → False    إذا: أي واحدة منهم False
```

خطوات التحقق:

1. خذ كل عنصر في Domain
2. عوّض في الجملة
3. إذا **كل** التعويضات True → النتيجة True
4. إذا **أي** تعويض False → النتيجة False

المكمم الوجودي (∃) - Existential Quantifier

القاعدة:

الجملة `∃X P(X)` صحيحة **إذا وُجدت قيمة واحدة على الأقل** لـ X تجعل P(X) صحيحة

```
Domain: {john, mary, kate}
Predicate: doctor(X)
∃X doctor(X) → True    إذا: doctor(john)=T أو doctor(mary)=T أو doctor(kate)=T
∃X doctor(X) → False   إذا: الكل False
```

خطوات التحقق:

1. ابحث عن **أي** عنصر في Domain
2. عوّض في الجملة
3. إذا **أي** تعويض True → النتيجة True
4. إذا **كل** التعويضات False → النتيجة False

🔥 مقارنة مهمة: ∃ vs ∀

الخاصية	∀X (للجميع)	∃X (يوجد)
True إذا	**كل** التعويضات True	**على الأقل واحد** True
False إذا	**أي واحد** False	**كل** التعويضات False
الصعوبة	يحتاج فحص الجميع	يكفي إيجاد واحد

📌 القسم الثالث: المفاهيم المهمة

🔷 المتغير الصوري (Dummy Variable)

تعريف: اسم المتغير غير مهم، يمكن تغييره دون تغيير المعنى

أمثلة متكافئة:

```text
∀X human(X) ≡ ∀Y human(Y)
∃X doctor(X) ≡ ∃Z doctor(Z)
```

شرط: تغيير الاسم في **كل** مواضعه

◆ النطاق (Scope)

تعريف: المدى الذي يؤثر فيه Quantifier على المتغير

مثال:

```text
∀X (human(X) → mortal(X))
     └──────────────────┘
         Scope of ∀X
```

الأقواس تحدد النطاق!

◆ المتغير الحر والمقيد (Free & Bound Variables) 🔥

المتغير المقيد (Bound Variable)

تعريف: متغير واقع تحت تأثير Quantifier

أمثلة:

```text
∀X human(X)          ← X مقيد (bound)
∃Y doctor(Y)         ← Y مقيد (bound)
∀X (parent(X, Y))    ← X مقيد، Y حر
```

◆ الجملة المغلقة (Closed Sentence)

تعريف: جملة **كل** متغيراتها مقيدة (لا توجد متغيرات حرة)

الجملة	النوع	السبب
`∀X human(X)`	✅ Closed	كل المتغيرات مقيدة
`∃X ∀Y likes(X, Y)`	✅ Closed	X و Y مقيدان
`likes(X, john)`	❌ Open	X حر
`∀Y parent(X, Y)`	❌ Open	X حر

أهمية: فقط الجمل المغلقة يمكن تحديد قيمة صدقها بشكل نهائي!

📌 القسم الرابع: مشكلة عدم القابلية للقرار (Undecidability)

🔷 المشكلة

السؤال: هل يمكن دائماً تحديد إذا كانت جملة True أو False؟

الإجابة: ❌ **لا!** (في حالات معينة)

🔷 السبب

Domain	Quantifier	المشكلة
Finite (محدود)	∀X أو ∃X	✅ يمكن فحص كل العناصر
Infinite (لا نهائي)	∀X	❌ مستحيل فحص عدد لا نهائي!
Infinite (لا نهائي)	∃X	🟡 يمكن (إذا وجدنا مثال واحد)

مثال:

```text
Domain = {1, 2, 3, 4, ...}  (جميع الأعداد الطبيعية)

∀X (X > 0)  → ❌ كيف نفحص عدد لا نهائي من الأعداد؟
```

🔷 الخلاصة

Predicate Calculus = Undecidable (غير قابل للقرار)

معنى ذلك:

- لا يوجد algorithm عام يضمن الوصول لإجابة في وقت محدود
- بعكس Propositional Logic (قابل للقرار)

📌 القسم الخامس: منطق الرتبة الأولى (First-Order Logic)

🔷 التعريف

First-Order Predicate Calculus:

Quantifiers (∀, ∃) تطبق **فقط** على المتغيرات التي تمثل **أشياء/كائنات**

🔷 ما هو مسموح؟

مسموح ✅	غير مسموح ❌
`∀X human(X)`	`∀P (P(john))`
`∃X doctor(X)`	`∃likes (likes(john, mary))`
`∀X ∃Y parent(X, Y)`	`∀father (father(X))`

السبب:

في First-Order، لا نكمّم على **Predicates** أو **Functions** نفسها، فقط على **Objects**

🔷 ما هو Second-Order Logic؟

Second-Order Logic: يسمح بتكميم Predicates والـ Functions

مثال:

```text
∀P ∃X P(X)  ← تكميم على Predicate (غير مسموح في First-Order)
```

ملاحظة: المنهج يركز على **First-Order فقط** ✅

📝 أمثلة محلولة

مثال 1: تحديد قيمة صدق جملة

Domain: `{john, mary, kate}`

Interpretation:

- `student(john) = T`
- `student(mary) = T`
- `student(kate) = F`

السؤال: ما قيمة `∀X student(X)`؟

الحل:

```text
∀X student(X)
→ student(john) ∧ student(mary) ∧ student(kate)
→ T ∧ T ∧ F
→ False ❌
```

الجواب: False (لأن kate ليست student)

مثال 2: Existential Quantifier

نفس Domain والـ Interpretation

السؤال: ما قيمة `∃X student(X)`؟

الحل:

```text
∃X student(X)
→ هل يوجد على الأقل واحد student؟
→ student(john) = T ✅
→ True
```

الجواب: True (لأن john طالب)

مثال 3: تحديد Free و Bound Variables

الجملة: `∀X (parent(X, Y) → loves(X, Y))`

التحليل:

المتغير	النوع	السبب
X	Bound	واقع تحت ∀X
Y	Free	لا يوجد quantifier له

النتيجة: جملة **Open** (غير مغلقة)

مثال 4: Scope

الجملة: `∀X (human(X) → ∃Y parent(Y, X))`

تحديد Scope:

```text
∀X (human(X) → ∃Y parent(Y, X))
    └──────────────────────────┘  ← Scope of ∀X
                   └──────────┘   ← Scope of ∃Y
```

مثال 5: جملة معقدة

Domain: `{1, 2, 3}`

Interpretation:

- `even(2) = T`
- `even(1) = F`
- `even(3) = F`

السؤال: ما قيمة `∃X (even(X) ∧ X > 1)`؟

الحل:

```text
نبحث عن x حيث: even(x)=T و x>1

X=1: even(1)=F → لا
X=2: even(2)=T نعم ✅ → 1<2 و!
X=3: even(3)=F → لا

النتيجة: True (وجدنا X=2)
```

الجواب: True

🎯 ملخص سريع للحفظ

Interpretation:

```text
Domain + تخصيص للثوابت/الدوال/المحمولات
```

Truth Values:

```text
∀X P(X) → True إذا P(X) صحيح للجميع
∃X P(X) → True إذا P(X) صحيح لواحد على الأقل
```

Variables:

```text
Bound   = تحت تأثير quantifier
Free    = خارج تأثير quantifier
Closed  = كل المتغيرات bound
```

First-Order Logic:

```text
Quantifiers على Objects فقط
ممنوع: ∀P, ∃likes
```

Undecidability:

```text
Domain لا نهائي + ∀X = مشكلة!
```

Inference Rules & Unification (قواعد الاستنتاج والتوحيد)

📌 مقدمة: ما هو الاستدلال (Inference)؟

الاستدلال: عملية استنتاج جمل جديدة (نتائج) من جمل موجودة (مقدمات)

المشكلة	الحل
لا يمكن فحص كل Interpretations (قد تكون لا نهائية)	استخدام قواعد استدلال ميكانيكية
كيف نضمن صحة النتائج؟	شرطا **Soundness** و **Completeness**

📌 القسم الأول: خصائص قواعد الاستدلال

🔷 السليم (Sound)

تعريف: القاعدة **سليمة** إذا كانت كل نتيجة تنتجها **صحيحة منطقياً**

```text
إذا كانت المقدمات صحيحة
+ القاعدة Sound
──────────────────
النتيجة صحيحة ضماناً ✅
```

مثال:

- **المقدمات صحيحة:** "كل البشر فانون" + "سقراط بشر"
- **القاعدة** Sound: Modus Ponens
- **النتيجة:** "سقراط فانٍ" ← **صحيحة ضماناً** ✅

أهمية: نظام Sound لن ينتج معلومات خاطئة أبداً!

🔷 الكامل (Complete)

تعريف: القاعدة **كاملة** إذا كانت قادرة على استنتاج **كل** النتائج الصحيحة الممكنة

```text
إذا كانت نتيجة صحيحة منطقياً
+ القاعدة Complete
──────────────────
يمكن استنتاجها بالقاعدة ✅
```

مثال:

- إذا كانت "سقراط فانٍ" **نتيجة منطقية** من المقدمات
- ونظامنا **Complete**
- إذن يمكن للنظام **استنتاجها** (لن نفقدها)

أهمية: نظام Complete لن يفقد أي معلومات صحيحة!

🔥 المقارنة الشاملة

الخاصية	Sound	Complete
المعنى	كل ما ينتجه **صحيح**	كل ما هو **صحيح** يمكن إنتاجه
الضمان	لا توجد نتائج خاطئة	لا توجد نتائج مفقودة
المشكلة إذا فُقدت	ينتج معلومات **خاطئة** ❌	يفقد معلومات **صحيحة** ❌
الأفضل	نظام **Sound + Complete** ✅	

📌 القسم الثاني: قواعد الاستدلال الأساسية

🔷 1. Modus Ponens (أهم قاعدة!) 🔥

الصيغة:

```text
المقدمات:   P → Q
            P
_____________
النتيجة:    Q
```

بالعربي:

- "إذا P إذن Q"
- "P صحيحة"
- **إذن:** "Q صحيحة"

أمثلة:

مثال 1: Propositional Logic

```text
المقدمات:
1. rain → wet_ground      (إذا أمطرت، الأرض مبللة)
2. rain                   (أمطرت)
──────────────────────
النتيجة:
3. wet_ground             (الأرض مبللة) ✅
```

مثال 2: Predicate Calculus

```text
المقدمات:
1. ∀X (human(X) → mortal(X))   (كل إنسان فانٍ)
2. human(socrates)              (سقراط إنسان)
──────────────────────────────
بعد Universal Instantiation:
   human(socrates) → mortal(socrates)
──────────────────────────────
النتيجة (Modus Ponens):
3. mortal(socrates)             (سقراط فانٍ) ✅
```

🔷 2. Modus Tollens

الصيغة:

```text
المقدمات:  P → Q
           ¬Q
──────────────
النتيجة:   ¬P
```

بالعربي:

- "إذا P إذن Q"
- "Q خاطئة"
- **إذن:** "P خاطئة"

مثال:

```text
المقدمات:
1. rain → wet_ground        (إذا أمطرت، الأرض مبللة)
2. ¬wet_ground              (الأرض ليست مبللة)
──────────────────────
النتيجة:
3. ¬rain                    (لم تمطر) ✅
```

🔷 3. And Elimination (∧-Elimination)

الصيغة:

```text
المقدمة:   P ∧ Q
──────────────
النتيجة:   P
           أو
           Q
```

بالعربي:

- إذا كان "P و Q" صحيح
- إذن "P" صحيح لوحده
- وأيضاً "Q" صحيح لوحده

مثال:

```text
المقدمة:
1. student(john) ∧ smart(john)
──────────────────────────────
النتيجة:
2. student(john)  ✅
3. smart(john)    ✅
```

◆ 4. And Introduction (∧-Introduction)

الصيغة:

```text
المقدمات:  P
           Q
──────────────
النتيجة:   P ∧ Q
```

بالعربي:

- إذا كان "P" صحيح
- وأيضاً "Q" صحيح
- إذن "P و Q" صحيح

مثال:

```text
المقدمات:
1. doctor(mary)
2. smart(mary)
──────────────
النتيجة:
3. doctor(mary) ∧ smart(mary)  ✅
```

◆ 5. Universal Instantiation (∀-Instantiation) 🔥

الصيغة:

```text
المقدمة:  ∀X P(X)
──────────────
النتيجة:  P(c)    (أي ثابت c حيث)
```

بالعربي:

- إذا كان "**لكل** X، P(X) صحيح"
- إذن "P(c)" صحيح لأي ثابت c

أمثلة:

مثال 1:

```text
المقدمة:
1. ∀X human(X) → mortal(X)
──────────────────────────
يمكن استنتاج:
2. human(john) → mortal(john)      ✅
3. human(mary) → mortal(mary)      ✅
4. human(socrates) → mortal(socrates) ✅
```

مثال 2:

```text
المقدمة:
1. ∀X (student(X) → studies(X))
──────────────────────────────
يمكن استنتاج:
2. student(ali) → studies(ali)     ✅
```

📌 القسم الثالث: التوحيد (Unification) 🔥

🔷 ما هو التوحيد؟

المشكلة:

- لدينا: `∀X (human(X) → mortal(X))`
- لدينا: `human(socrates)`
- **لا يمكن تطبيق Modus Ponens مباشرة!** (لأن `human(X)` ≠ `human(socrates)`)

الحل:

التوحيد (Unification): إيجاد **تعويضات (Substitutions)** تجعل جملتين **متطابقتين**

◆ التعويض (Substitution)

تعريف: قائمة بالتعويضات للمتغيرات

الرمز: `θ = {X/socrates}` تعني: "عوّض X بـ socrates"

مثال:

```text
الجملة الأصلية:  human(X)
التعويض:         θ = {X/socrates}
──────────────────────────────
النتيجة:         human(socrates)
```

◆ المُوحّد (Unifier)

تعريف: تعويض θ هو **unifier** لجملتين إذا جعلهما **متطابقتين** بعد التطبيق

مثال:

```text
الجملة 1:  human(X)
الجملة 2:  human(socrates)
──────────────────────────
Unifier:   θ = {X/socrates}
──────────────────────────
بعد التطبيق:
human(socrates) = human(socrates) ✅ (متطابقتان)
```

◆ Most General Unifier (MGU) 🔥

المشكلة:

قد يوجد **أكثر من unifier** للجملتين!

مثال:

```text
الجملة 1:  likes(X, Y)
الجملة 2:  likes(john, mary)
────────────────────────────
Unifiers ممكنة:
θ₁ = {X/john, Y/mary}      ✅
θ₂ = {X/john, Y/john}      ✅ (لكن أكثر تحديداً)
θ₃ = {X/Y, Y/mary}         ✅ (لكن ليست الأبسط)
```

الحل:

نختار **Most General Unifier (MGU)** = **أكثر تعويض عام** (أقل تحديداً)

✅ MGU: `θ₁ = {X/john, Y/mary}`

لماذا MGU؟

- يحافظ على **أقصى عمومية**
- يسمح باستخدامات أوسع في الاستدلالات المستقبلية

🔷 شروط التوحيد

1️⃣ المتغيرات يجب أن تكون معممة (Universally Quantified)

```text
✅ يمكن:  ∀X human(X)  → يمكن توحيد X
❌ لا يمكن: ∃X human(X) → لا يمكن توحيد X بحرية
```

2️⃣ Occurs Check ⚠️

تعريف: ممنوع توحيد متغير مع term يحتوي على المتغير نفسه

مثال:

```text
الجملة 1:  p(X)
الجملة 2:  p(f(X))
──────────────────
محاولة التوحيد:  θ = {X/f(X)}
──────────────────
❌ خطأ! (Occurs Check فشل)
```

السبب:

```text
X = f(X)
  = f(f(X))        (بالتعويض)
  = f(f(f(X)))     (تكرار لا نهائي!)
  = ...
```

ممنوع! ❌

🔷 خوارزمية التوحيد (Unification Algorithm)

الخطوات:

```text
1. إذا كانت الجملتان متطابقتان → θ = {} (تعويض فارغ) ✅
2. إذا كانت إحداهما متغير X:
   - تحقق من Occurs Check
   - إذا نجح → θ = {X/term}
3. إذا كانتا predicates/functions:
   - يجب أن يكون الاسم نفسه
   - يجب أن يكون العدد نفسه (arity)
   - وحّد كل معامل مع نظيره
4. وإلا → التوحيد فاشل ❌
```

📝 أمثلة محلولة

مثال 1: تطبيق Modus Ponens مع Unification

المعطيات:

```text
1. ∀X (bird(X) → can_fly(X))
2. bird(tweety)
```

المطلوب: استنتج معلومة جديدة

الحل:

```text
خطوة 1: Universal Instantiation
من (1): bird(X) → can_fly(X)

خطوة 2: Unification
نوحّد bird(X) مع bird(tweety)
θ = {X/tweety}

خطوة 3: تطبيق التعويض
bird(tweety) → can_fly(tweety)

خطوة 4: Modus Ponens
لدينا: bird(tweety) → can_fly(tweety)
لدينا: bird(tweety)
──────────────────────────────
النتيجة: can_fly(tweety) ✅
```

مثال 2: إيجاد MGU

السؤال: أوجد MGU لـ:

```text
likes(X, ice_cream)
likes(john, Y)
```

الحل:

```text
نوحّد المعامل الأول: X مع john
θ₁ = {X/john}

نوحّد المعامل الثاني: ice_cream مع Y
θ₂ = {Y/ice_cream}

MGU = {X/john, Y/ice_cream} ✅

التحقق:
likes(john, ice_cream) = likes(john, ice_cream) ✅
```

مثال 3: توحيد مع دوال

السؤال: أوجد MGU لـ:

```text
parent(father(X), X)
parent(father(john), john)
```

الحل:

```text
نوحّد المعامل الأول:
father(X) مع father(john)
→ X = john

نوحّد المعامل الثاني:
X مع john
→ X = john (متسق)

MGU = {X/john} ✅
```

مثال 4: Occurs Check

السؤال: هل يمكن توحيد؟

```text
p(X, f(X))
p(Y, Y)
```

الحل:

```text
نوحّد المعامل الأول: X مع Y
θ₁ = {X/Y}

نوحّد المعامل الثاني: f(X) مع Y
→ نحتاج: Y = f(X)
→ لكن X = Y (من الخطوة السابقة)
→ إذن: Y = f(Y)

Occurs Check: Y يظهر في f(Y)!
❌ التوحيد فاشل!
```

مثال 5: تطبيق Modus Tollens

المعطيات:

```text
1. ∀X (student(X) → attends_class(X))
2. ¬attends_class(ali)
```

المطلوب: استنتج

الحل:

```text
(1) من: student(X) → attends_class(X)

Unification: θ = {X/ali}
→ student(ali) → attends_class(ali)

Modus Tollens:
لدينا: student(ali) → attends_class(ali)
لدينا: ¬attends_class(ali)
──────────────────────────────────
النتيجة: ¬student(ali) ✅
(علي ليس طالباً)
```

🎯 ملخص سريع للحفظ

Inference Rules:

```text
Modus Ponens:     P→Q, P  ⊢ Q
Modus Tollens:    P→Q, ¬Q ⊢ ¬P
∧-Elimination:    P∧Q     ⊢ P (or Q)
∧-Introduction:   P, Q    ⊢ P∧Q
∀-Instantiation:  ∀X P(X) ⊢ P(c)
```

Soundness vs Completeness:

```text
Sound:    كل ما ينتجه صحيح
Complete: كل ما هو صحيح يمكن إنتاجه
```

Unification:

```text
MGU = Most General Unifier (الأكثر عمومية)
Occurs Check = ممنوع X = f(X)
```

خطوات تطبيق Modus Ponens في FOL:

```text
1. Universal Instantiation
2. Unification (إيجاد θ)
3. تطبيق التعويض
4. Modus Ponens
```

Unification (التوحيد)

1. ليش نحتاج التوحيد؟ (الفكرة ببساطة) 💡

بالمنطق القديم، P لازم تساوي P حتى نحذفهم. لكن بـ $Predicate$ عدنا متغيرات.

- **المشكلة:** شلون تقنع الحاسبة إن $human(X)$ هي نفسها $human(socrates)$؟
- **الحل:** عملية **التوحيد (Unification)**. هي مجرد "عملية مطابقة" ندور بيها على قيمة للمتغير (X) تخلي الجملتين يصيرن "طابق الأصل".

---

2. الـ θ (التعويض): "قائمة التبديلات" 📝

التعويض هو "الفيلم" كله. نكتبه بهيج صيغة: $\theta = \{value/variable\}$.

- **معناها:** "شيل المتغير، وحط بمكانه هاي القيمة".
- **مثال:** لو $likes(X, pizza)$ نطبق عليها $\{ali/X\}$، تصير $likes(ali, pizza)$.

---

3. شروط اللعبة (قوانين التوحيد الـ 4) ⚖️

هاي اللي تجي عليها أسئلة الـ MCQ "الخبيثة":

1. **الاسم والعدد (Name & Arity):** لازم نفس اسم الدالة ونفس عدد المعاملات.
   - $likes(X, Y)$ و $hates(X, Y)$ 👈 **فشل** (الأسماء مختلفة).
   - $parent(X, Y)$ و $parent(X, Y, Z)$ 👈 **فشل** (العدد مختلف).
2. **الثوابت "خط أحمر":** ما يصير تبدل ثابت بثابت ثاني.
   - ali و $ahmed$ 👈 **فشل**.
3. **الاتساق (Consistency):** إذا عوضت X بـ ali بأول الجملة، لازم تعوضه ali بكل الجملة. ما يصير X يأخذ قيمتين بنفس الوقت.
4. **فحص التكرار (Occurs Check)** 🔥**:** ممنوع تعوض متغير بدالة هو موجود بداخلها.
   - X و $f(X)$ 👈 **فشل**. ليش؟ لأنك راح تدخل بـ `Infinite Loop`.

4. الـ MGU: "الموحد الذكي" 🧠

بالامتحان، ممكن تلقى أكثر من تعويض يوحد الجمل، بس أنت لازم تختار الـ **Most General Unifier**.

- **القاعدة:** لا تقيد المتغير إذا مو محتاج.
- **مثال:** لتوحيد $p(X, Y)$ مع $p(A, A)$:
  - خيار 1: $\{A/X, A/Y\}$ 👈 هذا **MGU** (لأن A بقت مجهولة وعامة). ✅
  - خيار 2: $\{ali/X, ali/Y, ali/A\}$ 👈 هذا "محدد" زيادة عن اللزوم. ❌

---

5. شلون تحل أي مثال خطوة بخطوة؟ (الخوارزمية المبسطة) 🛠️

بدل ما تحفظ الكود، احفظ هاي الخطوات:

1. **طابق الأسماء والعدد:** إذا اختلفوا، كله $FAIL$.
2. **امشي سره (من اليسار لليمين):** طابق أول عنصرين.
3. **سجل التعويض:** إذا لقيت متغير وثابت، سجل التعويض θ.
4. **حدث الجملة فوراً:** أي تعويض تطلعه، طبقه على "بقية" الجملة قبل ما تنتقل للعنصر الثاني.
5. **تأكد من الـ Occurs Check:** قبل ما تسجل أي تعويض، تأكد إن المتغير مو موجود بداخل القيمة.

اسئلة ممكن تجي:

النمط الأول: "هل يمكن التوحيد؟" (Succeed or Fail)

هذا السؤال هدفه يختبرك بقوانين "الرفض" السريع.

- **السؤال:** هل يمكن توحيد $likes(X, Y)$ مع $hates(john, mary)$؟
- **كيفية الحل:**
  1. باوع على الاسم: $likes$ مقابل $hates$.. **مختلفين!**
  2. القرار: **Fail** فوراً.
- **السؤال:** هل يمكن توحيد $p(X, Y)$ مع $p(a, b, c)$؟
- **كيفية الحل:**
  1. احسب المعاملات ($Arity$): الأولى 2، الثانية 3.. **مختلفين!**
  2. القرار: **Fail** فوراً.

النمط الثاني: "فخ الـ Occurs Check" (المفضل للأساتذة) ⚠️

هذا يجي حتى يغلطك ويشوفك منتبه لو لا.

- **السؤال:** أوجد الموحد (MGU) لـ $P(X)$ و $P(f(X))$.
- **كيفية الحل:**
  1. الاسم والعدد تمام.
  2. المتغير هو X، والقيمة هي $f(X)$.
  3. اسأل نفسك: هل X موجود داخل $f(X)$؟ **نعم!**
  4. القرار: **Fail** (بسبب الـ $Occurs-Check$). الحاسبة لو حاولت تحله راح تدخل بـ `Infinite Loop` .

النمط الثالث: "التوحيد المتسلسل" (Chain of Substitutions)

هنا يختبرك بشلون تطبق التعويض وتحدث الجملة.

- **السؤال:** وحّد $parent(X, father(X))$ مع $parent(john, Y)$.
- **كيفية الحل (خوارزمية المسطرة):**
  1. طابق أول عنصرين: X مع $john$.. سجل التعويض الأول $\theta_1 = \{john/X\}$.
  2. **حدث الجملة فوراً:** شيل كل X وحط $john$ بالباقي.
  3. صارت الجملة الثانية: طابق $father(john)$ مع Y.
  4. سجل التعويض الثاني: $\theta_2 = \{father(john)/Y\}$.
- **النتيجة النهائية:** $\theta = \{john/X, father(john)/Y\}$.

النمط الرابع: "تحدي الاتساق" (Consistency Test)

هذا يختبر ذكاءك في حال تكرر المتغير بأكثر من مكان.

- **السؤال:** هل يمكن توحيد $likes(X, X)$ مع $likes(ali, ahmed)$؟
- **كيفية الحل:**
  1. طابق أول X مع ali.. سجل $\theta = \{ali/X\}$.
  2. حدث الجملة: صارت $likes(ali, ali)$ مقابل $likes(ali, ahmed)$.
  3. طابق العنصر الثاني: ali مقابل $ahmed$.. **فشل!** (لأن الثوابت لا تتطابق).
- **القرار: Fail.** (لأن المتغير X لا يمكن أن يأخذ قيمتين مختلفتين بنفس الوقت).

💡 **جدول "خطة الهجوم" للامتحان:**

الحالة	النتيجة	ملاحظة
اسمين مختلفين	**Fail**	$p(X) \neq q(X)$
عدد معاملات مختلف	**Fail**	$p(X) \neq p(X, Y)$
ثابت مع ثابت مختلف	**Fail**	$ali \neq ahmed$
متغير داخل دالة تحمله	**Fail**	X داخل $f(X)$ $(Occurs - Check)$
متغير مع ثابت	**Success**	سجل التعويض وحدث الباقي ✅
متغير مع متغير	**Success**	سجل التعويض (أكثر عمومية) ✅